Установка необходимых пакетов:

```
python -m pip install fastapi[all] uvicorn
```

Код для тестов:

```python
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from typing import List, Optional
import sqlite3

app = FastAPI()

def init_db():
    conn = sqlite3.connect('auction.db')
    cursor = conn.cursor()
    cursor.execute("""
    CREATE TABLE IF NOT EXISTS items (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        description TEXT,
        price REAL NOT NULL
    )
    """)
    conn.commit()
    conn.close()

init_db()

class Item(BaseModel):
    id: Optional[int] = None
    name: str
    description: Optional[str] = None
    price: float

@app.post("/items/", response_model=Item)
def create_item(item: Item):
    conn = sqlite3.connect('auction.db')
    cursor = conn.cursor()
    cursor.execute("""
    INSERT INTO items (name, description, price)
    VALUES (?, ?, ?)
    """, (item.name, item.description, item.price))
    item.id = cursor.lastrowid
    conn.commit()
    conn.close()
    return item

@app.get("/items/", response_model=List[Item])
def get_items():
```

```
    conn = sqlite3.connect('auction.db')
    cursor = conn.cursor()
    cursor.execute('SELECT * FROM items')
    rows = cursor.fetchall()
    conn.close()
    return [Item(id=row[0], name=row[1], description=row[2],
price=row[3]) for row in rows]

@app.get("/items/{item_id}", response_model=Item)
def get_item(item_id: int):
    conn = sqlite3.connect('auction.db')
    cursor = conn.cursor()
    cursor.execute('SELECT * FROM items WHERE id = ?', (item_id,))
    row = cursor.fetchone()
    conn.close()
    if row is None:
        raise HTTPException(status_code=404, detail="Item not found")
    return Item(id=row[0], name=row[1], description=row[2],
price=row[3])
```

Команда которой запускается сервер:

uvicorn main:app –reload

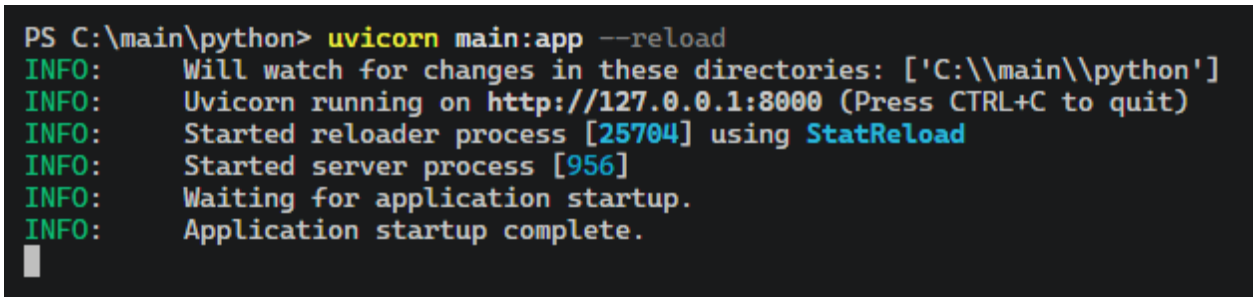


Рисунок 1 – Запуск сервера

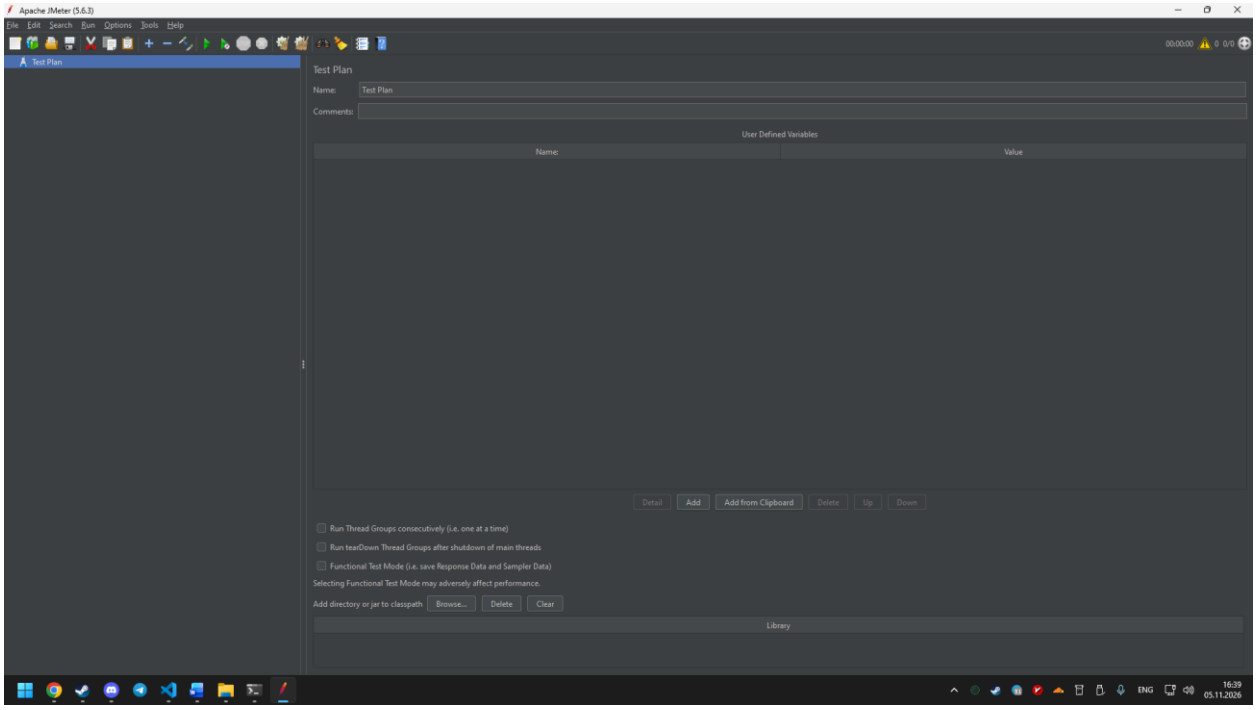


Рисунок 2 – Программа Apache JMeter

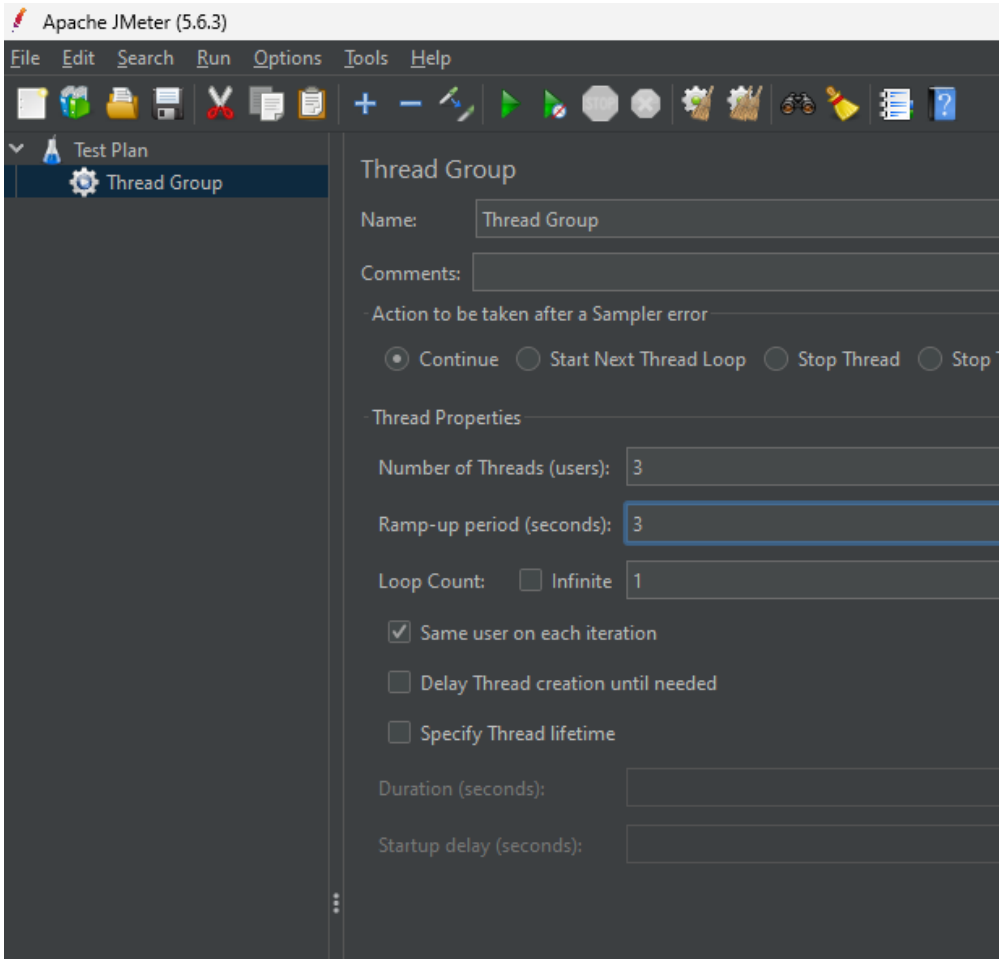


Рисунок 3 – Добавление Thread Group

Количество пользователь будет 3, а период времени 3 секунды.

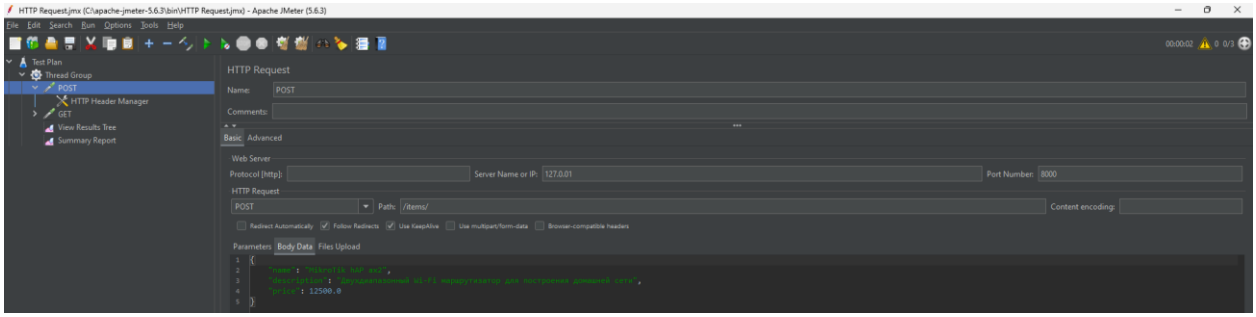


Рисунок 4 – Создание POST-запроса

В качестве тестовых данных используется сетевой маршрутизатор MikroTik hAP ax2. Запрос передаёт на сервер информацию о наименовании оборудования, его характеристиках и стоимости в формате JSON.

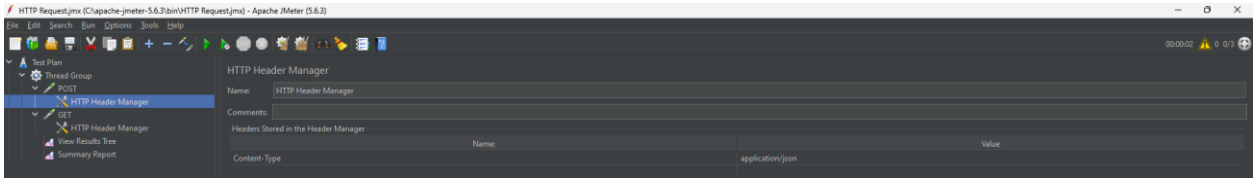


Рисунок 5 – Создание HTTP Header Manager для метода POST

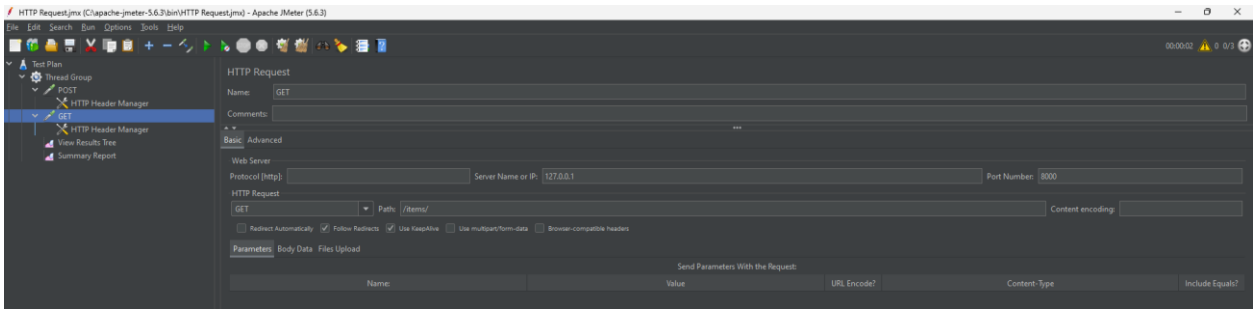


Рисунок 6 – Создание GET запроса

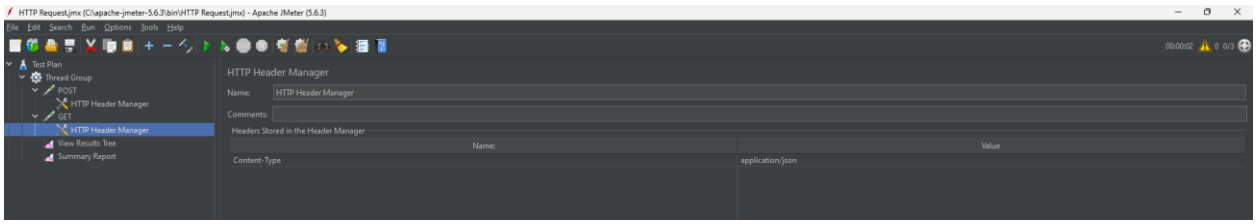


Рисунок 7 - Создание HTTP Header Manager для метода GET

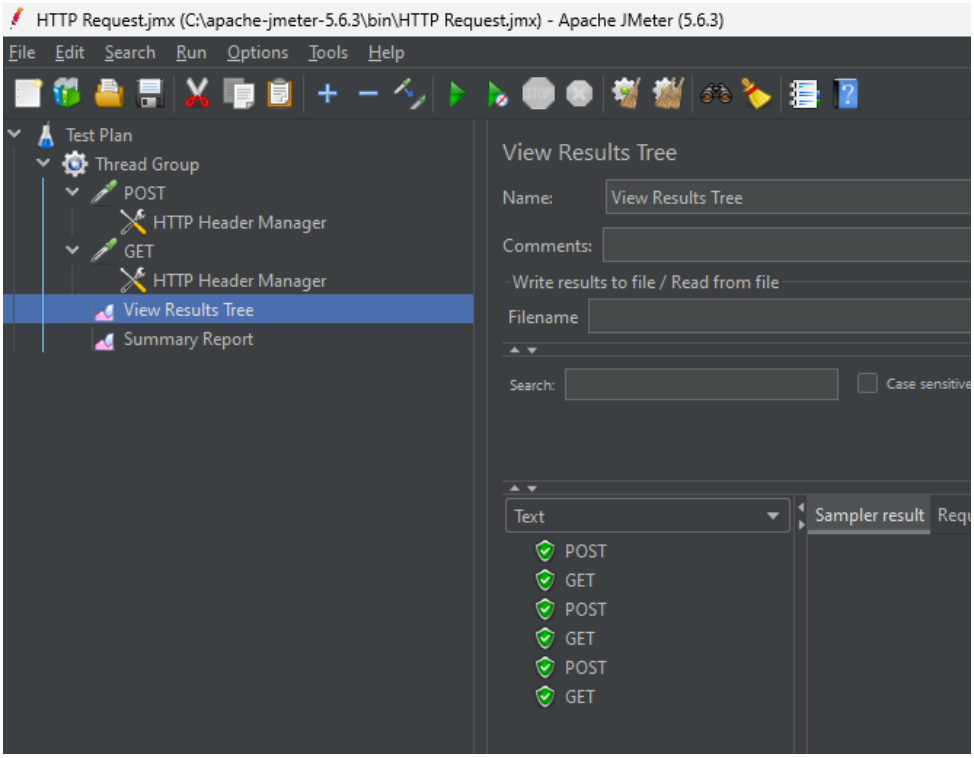


Рисунок 8 – Результаты успешно отправленных запросов в Jmeter

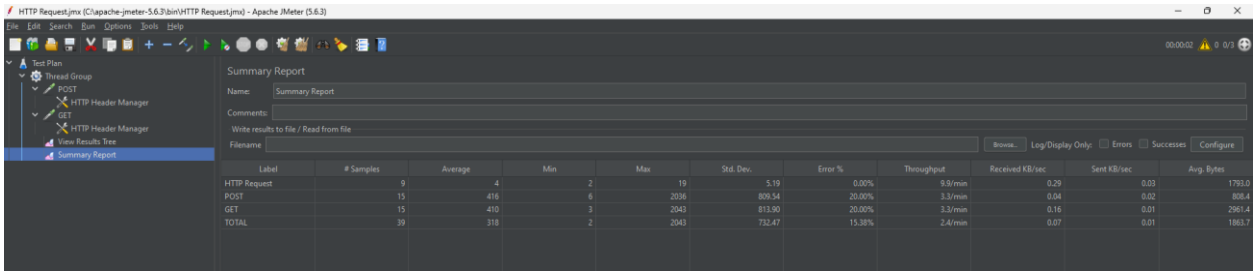


Рисунок 9 – Сводная таблица

```
PS C:\main\python> uvicorn main:app --reload
INFO:     Will watch for changes in these directories: ['C:\\main\\python']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [24084] using StatReload
INFO:     Started server process [28604]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     127.0.0.1:51415 - "POST /items/ HTTP/1.1" 200 OK
INFO:     127.0.0.1:51416 - "GET /items/ HTTP/1.1" 200 OK
INFO:     127.0.0.1:57435 - "POST /items/ HTTP/1.1" 200 OK
INFO:     127.0.0.1:57436 - "GET /items/ HTTP/1.1" 200 OK
INFO:     127.0.0.1:56522 - "POST /items/ HTTP/1.1" 200 OK
INFO:     127.0.0.1:56523 - "GET /items/ HTTP/1.1" 200 OK
```

Рисунок 10 – Результаты успешно отправленных запросов в CMD

Теперь надо сделать специально ошибочные GET и POST запросы.

Сначала для POST передадим данные запроса без обязательного параметра price.

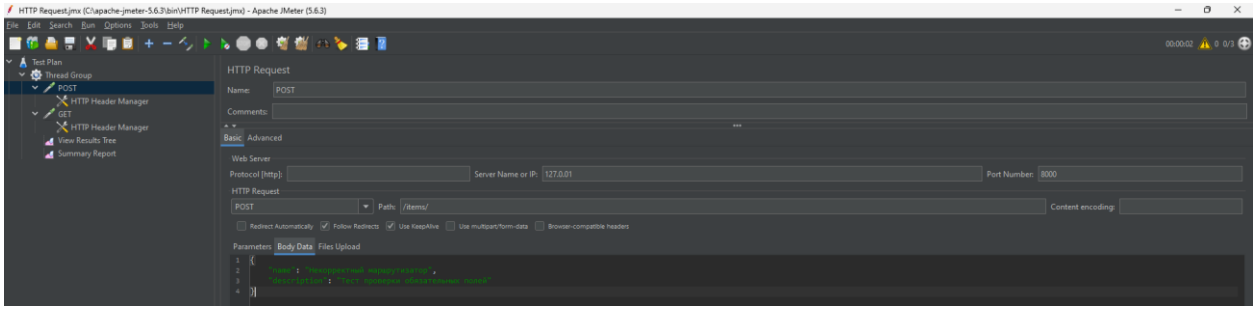


Рисунок 11 – Создание ошибочного POST запроса

Для GET запроса укажем в пути несуществующий id товара

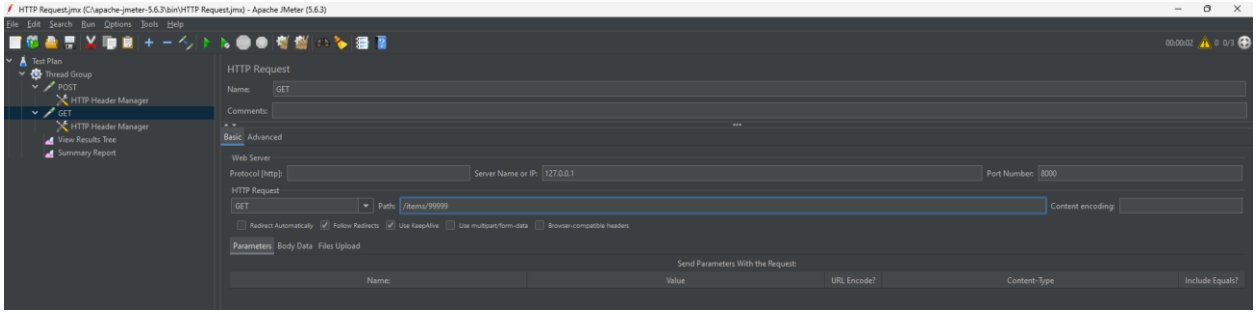


Рисунок 12 – Создание ошибочного GET запроса

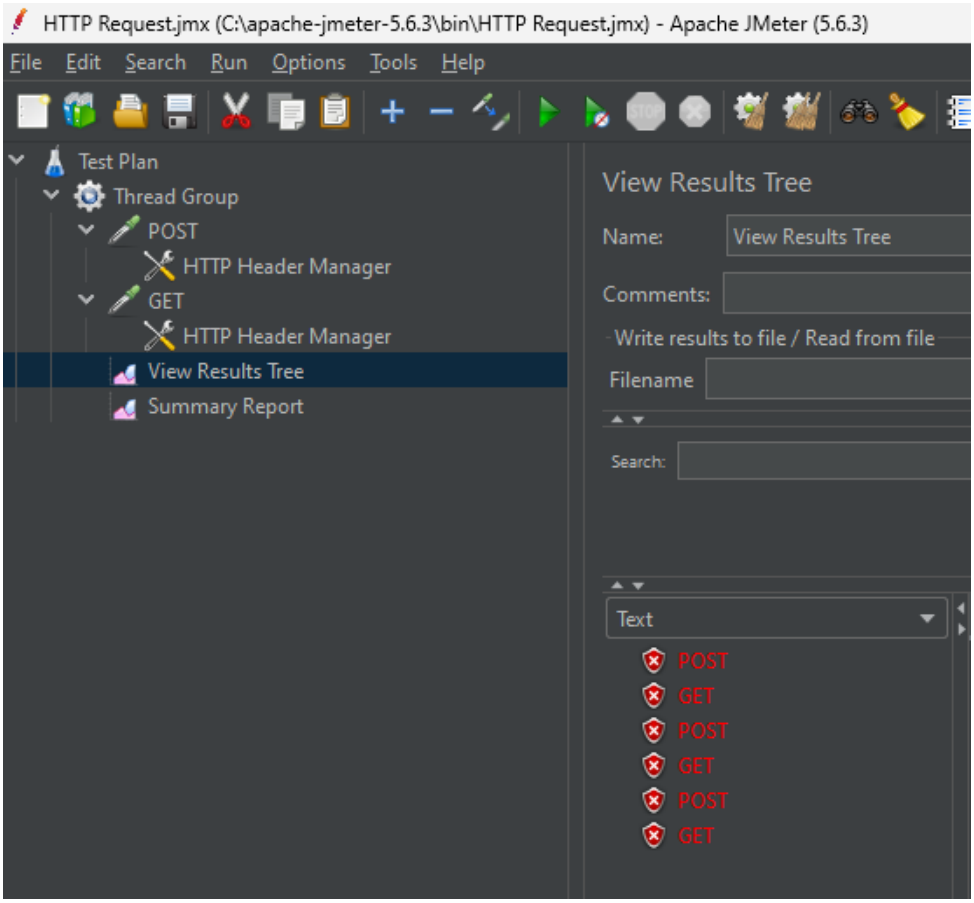


Рисунок 13 - Результаты ошибочно отправленных запросов в Jmeter

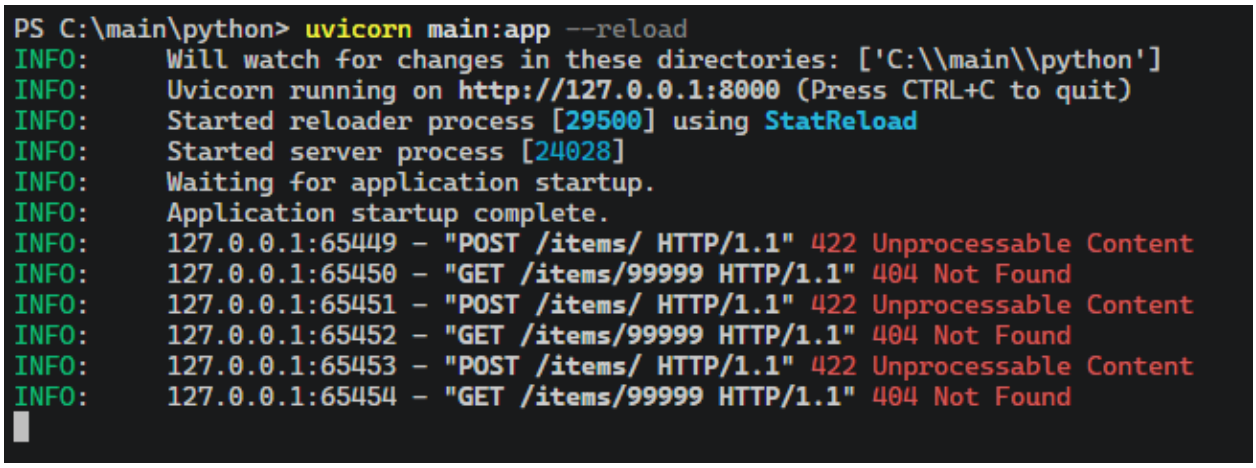


Рисунок 14 - Результаты ошибочно отправленных запросов в CMD